Week 2: Bash Scripting and Shell Environments

For Cloud ML Tasks

NMTAFE - Certificate IV Data Science and AI

Building automation skills for professional ML workflows

Learning Objectives

- Master essential Bash commands and scripting techniques for ML automation
- Create shell scripts for file management, data processing, and system operations
- Understand shell environments and their role in cloud-based ML workflows
- Integrate Bash automation with Python ML scripts for end-to-end pipelines

Why Bash Scripting for ML Engineers?

Industry Reality:

- **90% of ML infrastructure** runs on Linux/Unix systems
- **Cloud platforms** (Azure, AWS, GCP) use shell interfaces for automation
- **DevOps and MLOps** pipelines rely heavily on shell scripting
- **Reproducible workflows** require automated setup and execution

"A skilled ML engineer can automate their entire development pipeline with well-written shell scripts"

Shell vs. Graphical Interfaces

Graphical Interface (GUI):

- Point and click operations
- Good for individual tasks
- Hard to automate or reproduce
- Limited remote access capabilities

Shell Interface (CLI):

- Text-based commands
- Perfect for automation and scripting
- Easily reproducible and shareable
- Essential for remote cloud work

Essential Bash Commands for ML

File and Directory Operations:

```
ls -la                          # List files with details
mkdir ml_project                # Create directory
cd ml_project                   # Change directory
cp data.csv backup/             # Copy files
mv old_model.pt models/         # Move files
rm temp_files/*                 # Remove files
find . -name "*.py"             # Find Python files
```

Process and System Management:

```
ps aux                          # List running processes
top                             # Monitor system resources
kill -9 1234                    # Terminate process
df -h                           # Check disk space
```

Text Processing for Data Scientists

Viewing and Analyzing Files:

```
head -n 10 dataset.csv      # First 10 lines
tail -f training.log        # Follow log file updates
wc -l dataset.csv           # Count lines
grep "error" log.txt        # Find specific text
sort scores.txt             # Sort data
uniq users.txt              # Remove duplicates
cut -d',' -f1 data.csv      # Extract columns
```

Practical Example: Quickly analyze CSV structure without loading into Python

Your First Bash Script: ML Project Setup

```
#!/bin/bash
# ml_setup.sh - Initialize ML project structure

PROJECT_NAME="pytorch_experiment"
echo "Setting up ML project: $PROJECT_NAME"

# Create directory structure
mkdir -p $PROJECT_NAME/{data,models,notebooks,scripts,logs}

# Download sample dataset
echo "Downloading sample data..."
wget -O $PROJECT_NAME/data/iris.csv \
  https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data

# Create basic Python requirements file
cat > $PROJECT_NAME/requirements.txt << EOF
torch>=2.0.0
pandas>=1.5.0
numpy>=1.21.0
matplotlib>=3.5.0
jupyter>=1.0.0
EOF

echo "Project setup complete!"
```

Running and Managing Scripts

Make Script Executable:

```
chmod +x ml_setup.sh          # Add execute permission
./ml_setup.sh                 # Run the script
```

Script Variables and Parameters:

```
#!/bin/bash
PROJECT_NAME=$1               # First command line argument
DATA_URL=$2                   # Second argument

# Usage: ./setup.sh my_project http://data.url
if [ -z "$PROJECT_NAME" ]; then
    echo "Usage: $0 <project_name> <data_url>"
    exit 1
fi
```

Environment Variables and Configuration

Setting Environment Variables:

```
export CUDA_VISIBLE_DEVICES=0,1      # Specify GPU devices
export PYTORCH_CUDA_ALLOC_CONF=max_split_size_mb:128
export ML_DATA_PATH="/home/user/datasets"
export MODEL_CHECKPOINT_DIR="/models/checkpoints"
```

Reading Variables in Scripts:

```
#!/bin/bash
echo "Using GPUs: $CUDA_VISIBLE_DEVICES"
echo "Data path: $ML_DATA_PATH"

# Use default if variable not set
DATA_PATH=${ML_DATA_PATH:-"/default/data/path"}
```

Bash + Python Integration

Python Script Launcher:

```
#!/bin/bash
# train_model.sh - Launch Python training with proper environment

# Activate virtual environment
source venv/bin/activate

# Set environment variables
export CUDA_VISIBLE_DEVICES=0
export PYTHONPATH="${PYTHONPATH}:$(pwd)/src"

# Run training with parameters
python scripts/train.py \
  --data-path $ML_DATA_PATH \
  --epochs 100 \
  --batch-size 32 \
  --learning-rate 0.001

# Deactivate environment
```

Cloud Environment Setup Scripts

Azure VM Configuration Example:

```
#!/bin/bash
# azure_ml_setup.sh - Configure Azure VM for ML workloads

# Update system
sudo apt update && sudo apt upgrade -y

# Install Python and pip
sudo apt install python3-pip python3-venv -y

# Install CUDA drivers (for GPU VMs)
wget https://developer.download.nvidia.com/compute/cuda/repos/ubuntu2004/x86_64/cuda-ubuntu2004.pin
sudo mv cuda-ubuntu2004.pin /etc/apt/preferences.d/cuda-repository-pin-600

# Create ML user and setup directories
sudo useradd -m -s /bin/bash mluser
sudo mkdir -p /home/mluser/{projects,data,models}
sudo chown -R mluser:mluser /home/mluser/

echo "Azure ML VM setup complete!"
```

Automation with Cron Jobs

Schedule Regular Tasks:

```
# Edit crontab: crontab -e

# Run data backup every day at 2 AM
0 2 * * * /home/user/scripts/backup_data.sh

# Check model performance every hour
0 * * * * /home/user/scripts/monitor_model.sh

# Weekly cleanup of temporary files
0 0 * * 0 /home/user/scripts/cleanup_temp.sh
```

Example Backup Script:

```
#!/bin/bash
# backup_data.sh
DATE=$(date +%Y%m%d)
```

Error Handling and Logging

Robust Script Structure:

```
#!/bin/bash
# training_pipeline.sh - ML training with error handling

set -e    # Exit on any error
LOG_FILE="/logs/training_$(date +%Y%m%d_%H%M%S).log"

# Function for logging
log() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" | tee -a $LOG_FILE
}

# Main execution with error handling
main() {
    log "Starting ML training pipeline"

    if [ ! -f "data/train.csv" ]; then
        log "ERROR: Training data not found"
        exit 1
    fi

    log "Preprocessing data..."
    python preprocess.py || { log "ERROR: Preprocessing failed"; exit 1; }

    log "Training model..."
    python train.py || { log "ERROR: Training failed"; exit 1; }

    log "Training pipeline completed successfully"
}
```

File Processing and Data Pipeline Automation

CSV Data Processing Pipeline:

```
#!/bin/bash
# process_ml_data.sh - Automated data processing

INPUT_DIR="/data/raw"
OUTPUT_DIR="/data/processed"

# Create output directory
mkdir -p $OUTPUT_DIR

# Process all CSV files
for csv_file in $INPUT_DIR/*.csv; do
    filename=$(basename "$csv_file" .csv)

    echo "Processing $filename..."

    # Remove empty lines and sort by first column
    sed '/^$/d' "$csv_file" | sort -t',' -k1 > "$OUTPUT_DIR/${filename}_processed.csv"

    # Generate summary statistics
    echo "File: $filename" >> "$OUTPUT_DIR/summary.txt"
    echo "Records: $(wc -l < "$OUTPUT_DIR/${filename}_processed.csv")" >> "$OUTPUT_DIR/summary.txt"
    echo "---" >> "$OUTPUT_DIR/summary.txt"
done
```

Monitoring and System Health Checks

Resource Monitoring Script:

```
#!/bin/bash
# monitor_system.sh - Check system resources for ML workloads

# Check GPU utilization
if command -v nvidia-smi &> /dev/null; then
    echo "=== GPU Status ==="
    nvidia-smi --query-gpu=name,memory.total,memory.used,utilization.gpu --format=csv
fi

# Check CPU and Memory
echo "=== CPU/Memory Status ==="
echo "CPU Usage: $(top -bn1 | grep "Cpu(s)" | awk '{print $2}' | cut -d'%' -f1)%"
echo "Memory Usage: $(free | grep Mem | awk '{printf("%.2f%%", $3/$2 * 100.0)}')"

# Check disk space
echo "=== Disk Usage ==="
df -h | grep -E '^/dev/'

# Check running ML processes
```

Shell Scripting Best Practices

Code Quality Standards:

```
#!/bin/bash
# Always include shebang line

# Use set flags for safety
set -euo pipefail   # Exit on error, undefined vars, pipe failures

# Document your script
# Purpose: This script trains ML models
# Usage: ./train.sh <model_type> <data_path>
# Author: Your Name

# Use meaningful variable names
MODEL_TYPE=$1
DATA_PATH=$2
TIMESTAMP=$(date +%Y%m%d_%H%M%S)

# Validate inputs
if [ $# -ne 2 ]; then
    echo "Usage: $0 <model_type> <data_path>"
```

Practical Exercise: Complete ML Automation Script

Your Challenge: Create a script that:

1. Sets up a new ML project directory structure
2. Downloads a dataset from a URL
3. Installs Python requirements
4. Runs a simple data validation check
5. Logs all operations with timestamps
6. Handles errors gracefully

Template Structure:

```
#!/bin/bash
# TODO: Add proper error handling
# TODO: Add logging functionality
```

Cloud Integration: Azure CLI Basics

Azure VM Management:

```
# Login to Azure
az login

# Create resource group
az group create --name ml-resources --location australiaeast

# Create VM
az vm create \
  --resource-group ml-resources \
  --name ml-vm-01 \
  --image UbuntuLTS \
  --size Standard_NC6 \
  --admin-username azureuser \
  --generate-ssh-keys

# Connect to VM
ssh azureuser@<vm-ip-address>
```

Troubleshooting Common Issues

Permission Problems:

```
chmod +x script.sh          # Make executable
sudo chown user:user file   # Change ownership
```

Path Issues:

```
which python3                    # Find command location
echo $PATH                       # Check PATH variable
export PATH=$PATH:/new/path      # Add to PATH
```

Script Debugging:

```
bash -x script.sh           # Debug mode
set -x                      # Enable debugging in script
```

Integration with Next Week

Preparing for Week 3: Pseudocode and Script Design

- Today's Bash skills will combine with Python scripting
- You'll design complete ML automation workflows
- Focus shifts from individual commands to workflow logic
- Pseudocode helps plan complex, multi-step processes

Week 3 Preview: Translating workflow ideas into executable scripts

Key Takeaways

- ✅ **Shell scripting** automates repetitive ML tasks and reduces errors
- ✅ **Bash commands** are essential for cloud and Linux-based ML environments
- ✅ **Integration** between Bash and Python creates powerful ML pipelines
- ✅ **Best practices** ensure scripts are reliable and maintainable

"Master the shell, and you control your entire ML development environment"

Assignment and Next Steps

This Week's Practice:

1. Create the ML project setup script from today's exercise
2. Write a script that monitors system resources
3. Practice Azure CLI commands (if you have access)
4. Combine a Bash script with Python execution

Preparation for Week 3:

- Review any challenging concepts from today
- Think about multi-step ML workflows you'd like to automate
- Install git if not already available

Questions and Discussion

Reflection Questions:

- How could shell scripting improve your current development workflow?
- What ML tasks would benefit most from automation?
- When might you choose Bash over Python for a particular task?

Next Week: We'll design complete workflows using pseudocode before implementing them in scripts